



Data Structures And Algorithms

Name : Samreen Zia
Submitted to : Mr. Rehan Ahmed
Semester : BSE (3A)
Date : 03- Nov- 2025

LAB NO 7

Exercise 1:

A stack can be implemented using a linked list. The first node can serve as the 'top' of Stack and 'push' and 'pop' operations can be implemented by adding and removing nodes at the head of the linked list. Implement the Stack class using a linked list and provide all the standard member functions.

Program:

```
#include <iostream>
using namespace std;

class Stack {
private:
    struct node {
        int data;
        node *next;
    } *top;

public:
    Stack();
    ~Stack();
    bool isEmpty();
    void push(int value);
    int pop();
    int peek();
    void display();
};
```

```

// Constructor to initialize an empty stack
Stack::Stack() {
    top = nullptr;
}

// Destructor to free the memory allocated for the stack
Stack::~Stack() {
    node *temp;
    while (top != nullptr) {
        temp = top;
        top = top->next;
        delete temp;
    }
}

// Function to check if the stack is empty
bool Stack::isEmpty() {
    return top == nullptr;
}

// Function to push a new element onto the stack
void Stack::push(int value) {
    node *newNode = new node();
    newNode->data = value;
    newNode->next = top;
    top = newNode;
}

// Function to pop the top element from the stack
int Stack::pop() {
    if (isEmpty()) {
        cout << "Stack is empty. Cannot pop." << endl;
        return -1; // Return a sentinel value to indicate error
    }
    int value = top->data;
    node *temp = top;
    top = top->next;
    delete temp;
    return value;
}

// Function to peek at the top element of the stack
int Stack::peek() {

```

```

    if (isEmpty()) {
        cout << "Stack is empty." << endl;
        return -1; // Return a sentinel value to indicate error
    }
    return top->data;
}

// Function to display the elements of the stack
void Stack::display() {
    node *temp = top;
    while (temp != nullptr) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}

int main() {
    Stack stack;
    cout << "Is stack empty? " << (stack.isEmpty() ? "Yes" : "No") << endl;
    stack.push(10);
    stack.push(20);
    stack.push(30);
    cout << "Stack: ";
    stack.display(); // Output: 30 20 10
    cout << "Top element: " << stack.peek() << endl; // Output: 30
    int popped = stack.pop();
    cout << "Popped element: " << popped << endl; // Output: 30
    cout << "Stack after pop: ";
    stack.display(); // Output: 20 10
    return 0;
}

```

Output:

```

Is stack empty? Yes
Stack: 30 20 10
Top element: 30
Popped element: 30
Stack after pop: 20 10

```

Exercise 2

Implement simple (Linear) Queue through linked list.

Program:

```
#include <iostream>
using namespace std;

class Queue {
private:
    struct node {
        int data;
        node *next;
    } *front, *rear;

public:
    Queue();
    ~Queue();
    bool isEmpty();
    void enqueue(int value);
    int dequeue();
    int peek();
    void display();
};

// Constructor to initialize an empty queue
Queue::Queue() {
    front = rear = nullptr;
}

// Destructor to free the memory allocated for the queue
Queue::~~Queue() {
    node *temp;
    while (front != nullptr) {
        temp = front;
        front = front->next;
        delete temp;
    }
}

// Function to check if the queue is empty
bool Queue::isEmpty() {
    return front == nullptr;
}
```

// Function to add an element to the queue

```
void Queue::enqueue(int value) {
```

```
    node *newNode = new node();
```

```
    newNode->data = value;
```

```
    newNode->next = nullptr;
```

```
    if (rear == nullptr) {
```

```
        front = rear = newNode;
```

```
    } else {
```

```
        rear->next = newNode;
```

```
        rear = newNode;
```

```
    }
```

```
}
```

// Function to remove an element from the queue

```
int Queue::dequeue() {
```

```
    if (isEmpty()) {
```

```
        cout << "Queue is empty. Cannot dequeue." << endl;
```

```
        return -1; // Return a sentinel value to indicate error
```

```
    }
```

```
    int value = front->data;
```

```
    node *temp = front;
```

```
    front = front->next;
```

```
    if (front == nullptr) {
```

```
        rear = nullptr;
```

```
    }
```

```
    delete temp;
```

```
    return value;
```

```
}
```

// Function to peek at the front element of the queue

```
int Queue::peek() {
```

```
    if (isEmpty()) {
```

```
        cout << "Queue is empty." << endl;
```

```
        return -1; // Return a sentinel value to indicate error
```

```
    }
```

```
    return front->data;
```

```
}
```

// Function to display the elements of the queue

```
void Queue::display() {
```

```

    node *temp = front;
    while (temp != nullptr) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}

int main() {
    Queue queue;
    cout << "Is queue empty? " << (queue.isEmpty() ? "Yes" : "No") << endl;
    queue.enqueue(10);
    queue.enqueue(20);
    queue.enqueue(30);
    cout << "Queue: ";
    queue.display(); // Output: 10 20 30
    cout << "Front element: " << queue.peek() << endl; // Output: 10
    int dequeued = queue.dequeue();
    cout << "Dequeued element: " << dequeued << endl; // Output: 10
    cout << "Queue after dequeue: ";
    queue.display(); // Output: 20 30
    return 0;
}

```

Output:

```

Is queue empty? Yes
Queue: 10 20 30
Front element: 10
Dequeued element: 10
Queue after dequeue: 20 30

```

Exercise 3

Write the following C++ functions to realize the indicated functionality on a singly linked list of integers.

- A function that prints only the even-numbered nodes of the list.
- A function which takes two data values as arguments and swaps the node values containing them.
- A function that deletes the first half of the linked list nodes. You are required to first determine the total number of linked list nodes and then delete the first part of the linked list.

Program:

```
#include <iostream>
using namespace std;

class List {
private:
    struct node {
        int data;
        node *next;
    } *head;

public:
    List();
    ~List();
    void insert(int value);
    void printEvenNodes();
    void swap(int value1, int value2);
    void deleteFirstHalf();
    void display();
};

// Constructor to initialize an empty list
List::List() {
    head = nullptr;
}

// Destructor to free the memory allocated for the list
List::~List() {
    node *temp;
    while (head != nullptr) {
        temp = head;
        head = head->next;
        delete temp;
    }
}

// Function to insert a new node at the end of the list
void List::insert(int value) {
    node *newNode = new node();
    newNode->data = value;
    newNode->next = nullptr;

    if (head == nullptr) {
        head = newNode;
    }
}
```

```

    } else {
        node *temp = head;
        while (temp->next != nullptr) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}

```

// Function to print only the even-numbered nodes of the list

```

void List::printEvenNodes() {
    node *temp = head;
    int count = 1;
    while (temp != nullptr) {
        if (count % 2 == 0) {
            cout << temp->data << " ";
        }
        temp = temp->next;
        count++;
    }
    cout << endl;
}

```

// Function to swap the node values containing two given values

```

void List::swap(int value1, int value2) {
    if (value1 == value2) return;

    node *temp1 = nullptr;
    node *temp2 = nullptr;
    node *current = head;

    while (current != nullptr) {
        if (current->data == value1) {
            temp1 = current;
        } else if (current->data == value2) {
            temp2 = current;
        }
        current = current->next;
    }

    if (temp1 != nullptr && temp2 != nullptr) {
        int $temp = temp1->data;
        temp1->data = temp2->data;
        temp2->data = $temp;
    }
}

```



```
}  
}
```

```
// Function to delete the first half of the linked list nodes
```

```
void List::deleteFirstHalf() {
```

```
    int count = 0;
```

```
    node *temp = head;
```

```
    // Determine the total number of linked list nodes
```

```
    while (temp != nullptr) {
```

```
        count++;
```

```
        temp = temp->next;
```

```
    }
```

```
    // Calculate the number of nodes to delete
```

```
    int nodesToDelete = count / 2;
```

```
    // Delete the first half of the linked list nodes
```

```
    while (nodesToDelete > 0) {
```

```
        node *temp = head;
```

```
        head = head->next;
```

```
        delete temp;
```

```
        nodesToDelete--;
```

```
    }
```

```
}
```

```
// Function to display the elements of the list
```

```
void List::display() {
```

```
    node *temp = head;
```

```
    while (temp != nullptr) {
```

```
        cout << temp->data << " ";
```

```
        temp = temp->next;
```

```
    }
```

```
    cout << endl;
```

```
}
```

```
int main() {
```

```
    List list;
```

```
    list.insert(1);
```

```
    list.insert(2);
```

```
    list.insert(3);
```

```
    list.insert(4);
```

```
    list.insert(5);
```

```
    list.insert(6);
```

```
cout << "Original List: ";  
list.display(); // Output: 1 2 3 4 5 6  
  
cout << "Even-numbered nodes: ";  
list.printEvenNodes(); // Output: 2 4 6  
  
list.swap(2, 5);  
cout << "List after swapping 2 and 5: ";  
list.display(); // Output: 1 5 3 4 2 6  
  
list.deleteFirstHalf();  
cout << "List after deleting first half: ";  
list.display(); // Output: 4 2 6  
  
return 0;  
}
```

Output:

```
Original List: 1 2 3 4 5 6  
Even-numbered nodes: 2 4 6  
List after swapping 2 and 5: 1 5 3 4 2 6  
List after deleting first half: 4 2 6
```